

Déploiement TEFD — du repo vide à l'appli installée sur le téléphone du réceptionniste

Cette procédure couvre les **trois couches** à mettre en ligne, dans l'ordre où les monter :

1. **Baserow** — le back-office, source de vérité unique.
2. **GitHub Pages** — le site statique + la coquille PWA (installable Android / iPhone / tablette).
3. **Cloudflare Worker** — le proxy léger (résolution d'adresse à la saisie + token Baserow caché).

Aucune étape ne nécessite de carte bancaire (tous les paliers gratuits suffisent au volume d'une réception hôtelière).

0. Ce qui existe déjà / ce qui manque

Pièce	Statut
Schéma Baserow (7 tables)	✅ livré — <code>schema_baserow_tefd.md</code>
<code>export.mjs</code> (Baserow → <code>app_data.json</code>)	✅ livré et patché (<code>priorite_edito</code> inclus)
<code>precompute-walk.mjs</code> (temps de marche ORS)	✅ livré
<code>build-data.yml</code> (orchestration GitHub Actions)	✅ livré
Moteur de reco 4 signaux	✅ livré — <code>moteur-reco.js</code> + tests
Maquette cliquable (démon Club)	✅ livrée, testée (22/22, Chromium headless)
Manifest PWA + icônes + service worker	✅ livrés dans cette étape (<code>manifest.json</code> , <code>sw.js</code> , <code>icons/</code>)
Page de production réelle (<code>index.html</code>) qui lit <code>app_data.json</code>	🕒 pas encore construite — c'est la prochaine étape de code , après ce déploiement

Important : la maquette HTML actuelle contient encore les données de démo en dur (`const LIEUX = [...]`). La page de production reprendra le même squelette HTML/CSS/JS, mais remplacera ce bloc par un `fetch('./app_data.json')`. Cette procédure suppose que cette page existe sous `public/index.html` — à construire avant la mise en ligne réelle, ou à blanc pour valider la chaîne technique avant.

1. Baserow — créer le back-office

1. Créer un compte sur baserow.io (gratuit) ou une instance self-hosted.
 2. Créer une base, puis les **7 tables** décrites dans `schema_baserow_tefd.md` (Adresses, Événements, Parcours, Personas, Catégories, Suggestions, Hôtels), avec les champs et types exacts du document.
 3. Point de vigilance déjà identifié dans la spec du moteur : le **champ primaire de la table Personas doit être** `code` (01 à 08), pas le nom — c'est cette valeur que le moteur de reco compare.
 4. Remplir au moins quelques lignes "Publié" dans Adresses pour avoir un premier export non vide.
 5. Récupérer :
 - un **token API** (Paramètres du compte → Jetons API),
 - l'**ID numérique de chaque table** (visible dans l'URL de la table, ou via l'API `GET /api/database/tables/`)).
-

2. Dépôt GitHub — structure

```
tefd-app/
├── .github/workflows/build-data.yml
├── scripts/
│   ├── export.mjs
│   └── precompute-walk.mjs
├── public/                                ← tout ce qui est servi par GitHub Pages
│   ├── index.html                        ← la page de production (squelette de la
maquette + fetch JSON)
│   ├── manifest.json
│   ├── sw.js
│   ├── icons/
│   │   ├── icon-192.png
│   │   ├── icon-512.png
│   │   ├── icon-512-maskable.png
│   │   ├── apple-touch-icon.png
│   │   └── favicon-32.png
│   └── app_data.json                    ← généré automatiquement, ne pas éditer à la
main
```

`build-data.yml` écrit dans `public/app_data.json` (variable `OUT_PATH`) — la structure ci-dessus est celle qu'il attend déjà.

3. Secrets GitHub Actions

Dans le repo : **Settings** → **Secrets and variables** → **Actions** → **New repository secret**.

Ajouter exactement ces clés (ce sont celles que lit `build-data.yml`) :

Secret	Valeur
<code>BASEROW_TOKEN</code>	le token API Baserow
<code>BASEROW_API_URL</code>	vide si baserow.io ; sinon l'URL de l'instance self-hosted
<code>T_ADRESSES</code> , <code>T EVENEMENTS</code> , <code>T_PARCOURS</code> , <code>T_PERSONAS</code> , <code>T_CATEGORIES</code> , <code>T_HOTELS</code>	les 6 ID de tables Baserow correspondants
<code>ORS_API_KEY</code>	clé OpenRouteService (gratuite, à créer sur openrouteservice.org)

4. Premier run du pipeline de données

1. Onglet **Actions** du repo → workflow `build-data` → **Run workflow** (déclenchement manuel, `workflow_dispatch` est déjà prévu).
2. Vérifier les logs : `export.mjs` doit annoncer le nombre d'adresses/événements/parcours exportés, puis `precompute-walk.mjs` le nombre de trajets calculés.
3. Vérifier que `public/app_data.json` a été committé (un commit `data: rebuild app_data.json (date)` doit apparaître).

Si ça échoue ici, c'est presque toujours un nom de champ Baserow qui ne correspond pas exactement à celui attendu par `export.mjs` (`note_edito_fr`, `place_id_google`, `priorite_edito`...) — comparer avec `schema_baserow_tefd.md`.

5. La coquille PWA — ce qu'il faut ajouter à `index.html`

Trois livrables sont prêts à déposer dans `public/` : `manifest.json`, `sw.js`, et le dossier `icons/`. Il reste à câbler `index.html` :

Dans le `<head>`, avant les feuilles de style :

```
html
```

```
<link rel="manifest" href="./manifest.json">
<meta name="theme-color" content="#3a2a4d">
<link rel="icon" href="./icons/favicon-32.png" sizes="32x32">

<!-- iOS : balises spécifiques, Safari ignore le manifest pour ces aspects -->
<link rel="apple-touch-icon" href="./icons/apple-touch-icon.png">
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
<meta name="apple-mobile-web-app-title" content="TEFD">

<!-- Android / Chromium : équivalent non-standard mais inoffensif à garder -->
<meta name="mobile-web-app-capable" content="yes">
```

Juste avant la fermeture de `</body>`, ou en fin de script :

```
html

<script>
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => navigator.serviceWorker.register('./sw.
})
</script>
```

Piège n°1 — chemins relatifs. GitHub Pages sert le site sous

`https://<compte>.github.io/<repo>/`, pas à la racine du domaine. Tous les chemins (`manifest.json`), (`sw.js`), icônes, (`app_data.json`) doivent rester **relatifs** (`./...`), jamais en `/...` absolu — sinon ça casse dès qu'on n'est pas à la racine. Le manifest livré utilise déjà ce format.

Piège n°2 — mise à jour de la coquille. Une fois la PWA installée sur le téléphone d'un réceptionniste, le service worker sert la coquille depuis son cache. Si vous modifiez le HTML/CSS/JS plus tard, **incrémentez** `CACHE_VERSION` dans `sw.js` (`tefd-shell-v1` → `v2`)... — sinon l'appareil reste bloqué sur l'ancienne interface. Les *données*, elles, n'ont pas ce problème : `app_data.json` est volontairement exclu du cache (règle anti-hallucination), donc toujours servi frais sans rien à faire.

6. Activer GitHub Pages

Settings → Pages → Source : choisir la branche `main`, dossier `/public`. Sauvegarder. L'URL est généralement disponible en 1-2 minutes sous `https://<compte>.github.io/<repo>/`.

GitHub Pages est HTTPS par défaut — condition obligatoire pour qu'un service worker s'enregistre (aucune PWA n'est installable en HTTP).

7. Déployer le Worker Cloudflare

1. `npm install -g wrangler` puis `wrangler login`.
2. `wrangler init tefd-worker` (ou réutiliser un Worker existant).
3. Le code du Worker porte deux responsabilités (cf. décisions actées plus haut dans le projet) :
 - résoudre un `place_id` Google à la saisie d'une nouvelle adresse (Text Search, tier gratuit) ;
 - porter le token Baserow pour l'écran admin / la table Suggestions, sans jamais l'exposer côté client.
4. Variables d'environnement du Worker (`wrangler secret put <NOM>`) : `BASEROW_TOKEN`, `GOOGLE_API_KEY`.
5. `wrangler deploy` → note l'URL `https://tefd-worker.<compte>.workers.dev`, à utiliser dans l'écran admin de résolution d'adresse.

Pas de cold start, pas de carte bancaire, 100 000 requêtes/jour gratuites — largement au-dessus du besoin réel.

8. Recette par plateforme

L'installation d'une PWA ne se comporte **pas du tout pareil** sur iOS et sur Android — c'est la source n°1 de confusion au lancement. Voici l'état réel (vérifié à ce jour) :

Android (téléphone et tablette), navigateur Chrome / Edge / Samsung Internet

- Si le manifest + service worker + HTTPS + icônes sont corrects, Chrome **propose automatiquement** l'installation (bannière ou icône "+" dans la barre d'adresse).
- Sinon : menu : → « **Installer l'application** » ou « **Ajouter à l'écran d'accueil** ».
- Une fois installée, l'icône apparaît sur l'écran d'accueil et l'appli s'ouvre en plein écran (sans barre d'adresse), exactement comme une appli native.
- **Audit qualité** : ouvrir les DevTools → onglet *Lighthouse* → catégorie *PWA* → la case « Installable » doit passer au vert. C'est le test le plus fiable avant de présenter au Club.

iPhone / iPad, Safari

Le comportement est structurellement différent — **pas de bannière automatique sur iOS**, quelle que soit la qualité du manifest. C'est une limite délibérée d'Apple, pas un bug de l'appli.

Procédure manuelle, à montrer une fois aux équipes de réception :

1. Ouvrir le site **dans Safari** (impératif — voir piège ci-dessous).
2. Toucher l'icône **Partager** (carré avec flèche vers le haut) dans la barre d'outils.

3. Faire défiler et toucher « **Sur l'écran d'accueil** ».
4. Confirmer le nom, toucher « **Ajouter** ».
5. L'icône apparaît sur l'écran d'accueil ; au lancement, l'appli s'ouvre en plein écran, sans barre Safari.

Trois pièges spécifiques à iOS, à connaître avant qu'un hôtel ne signale un faux problème :

- **Doit passer par Safari.** Si le lien est ouvert depuis un navigateur intégré à une autre appli (Outlook, Teams, Instagram...), l'option « Sur l'écran d'accueil » est absente ou dégradée. Il faut ouvrir explicitement dans Safari.
- **La Navigation privée bloque l'installation.** Si Safari est en mode privé, l'option n'apparaît pas.
- **Les restrictions « Temps d'écran »** (Réglages → Temps d'écran → Contenu et confidentialité) peuvent aussi la masquer sur un appareil géré (iPad de prêt, par exemple).

Depuis iOS 17.4 (suite au règlement européen DMA), Chrome et Edge sur iPhone peuvent eux aussi proposer une installation via leur propre bouton de partage dans certaines régions — mais Safari reste le chemin universel et le plus fiable à recommander aux équipes.

Tablettes — la vraie règle à retenir

Une tablette suit **la règle de son système, pas de sa taille d'écran** : une tablette Android se comporte comme un téléphone Android (bannière automatique Chrome), un iPad se comporte comme un iPhone (installation manuelle via Safari). Il n'y a pas de troisième cas à gérer.

Point d'attention propre à cette appli : le point de rupture CSS actuel bascule en mise en page empilée (carte sous la liste) à 860px de largeur. Un iPad en **portrait** (souvent 768-834px) tombera dans ce mode empilé — à valider visuellement une fois la page de production en ligne, sur un vrai iPad si possible.

9. Checklist finale avant d'annoncer au Club

- ☐ `app_data.json` se régénère bien chaque nuit (vérifier l'historique des commits automatiques après 24h)
- ☐ Lighthouse → PWA → « Installable » au vert
- ☐ Installation testée sur un Android réel (pas seulement l'émulateur Chrome)
- ☐ Installation testée sur un iPhone réel via Safari (le simulateur iOS ne suffit pas toujours pour ce flux)
- ☐ Un iPad en orientation portrait affiché correctement
- ☐ Le Worker répond (tester la résolution d'une nouvelle adresse depuis l'écran admin)

- ☐ Les boutons « Y aller » / « Photos & avis » ouvrent bien Google Maps sans déclencher d'appel facturé (modèle tout-lien)
 - ☐ `CACHE_VERSION` du service worker documentée comme étape à incrémenter à chaque future mise à jour de code
-

10. Ce qui se passe après le lancement (cycle de vie)

- **Les données** se mettent à jour seules : Baserow → webhook ou cron quotidien (03:30 UTC, déjà dans `build-data.yml`) → nouveau `app_data.json` → servi frais à chaque ouverture, sans jamais toucher au cache du service worker. Aucune action humaine récurrente needed côté code.
- **Le code** (HTML/CSS/JS de l'appli, pas les données) ne change que si vous faites évoluer des fonctionnalités. Dans ce cas : modifier, commiter, **incrémenter** `CACHE_VERSION`, et c'est tout — les appareils déjà installés récupèrent automatiquement la nouvelle coquille au prochain lancement.
- **Le Worker** ne se redéploie que si son propre code change (rare — résolution `place_id`, gestion du token).